
Topics Covered:

- Exercises on Advanced Data Types:
 - Lists
 - Tuples
 - Sets
 - Dictionaries
- Class



A. Explanation: Tuples, Sets, List, and Dictionaries



List in Python

List = [10, 'Favtutor', 10, [5, 10, 15]]



List[0] List[1] List[2] List[3]

- ✓ **Ordered:** Items have defined order which cannot be changed
- ✓ **Mutable:** Items can be modified anytime
- ✓ **Allow duplicates:** Items with the same value is allowed

What are Lists in Python?

Python possesses a list as a data structure that is an ordered sequence of elements and mutable in nature. Each item or value that is inside of a list is called an element. Just as strings are defined as characters between quotes, lists are defined by having values between square brackets ([]) separated by commas.

For Example:

```
sample_list = ["Python", "is", "Fun"]  
print(sample_list)
```

Output

```
['Python', 'is', 'Fun']
```



Tuples in Python

T = (20, 'Jessa', 35.75, [30, 60, 90])

↑ ↑ ↑ ↑

T[0] T[1] T[2] T[3]

- ✓ **Ordered**: Maintain the order of the data insertion.
- ✓ **Unchangeable**: Tuples are immutable and we can't modify items.
- ✓ **Heterogeneous**: Tuples can contains data of types
- ✓ **Contains duplicate**: Allows duplicates data

What is a Tuple

Tuples are ordered collections of heterogeneous data that are unchangeable. Heterogeneous means tuples can store variables of all types.

Creating a tuple with the use of integers

Example:

```
1 | Tuple1 = (0, 1, 2, 3)
2 | print("\nTuple is:", Tuple1)
```

Python

Tuple is: (0, 1, 2, 3)



Set in Python

$S = \{ 20, 'Jessa', 35.75 \}$

- ✓ **Unordered:** Set doesn't maintain the order of the data insertion.
- ✓ **Heterogeneous:** Set can contains data of all types
- ✓ **Unique:** Set doesn't allows duplicates items

What is a Set?

A set is an unordered collection of unique elements enclosed in curly braces (`{}`). It does not allow duplicate elements and does not preserve the order of insertion. Sets are useful when you need to perform membership checks or remove duplicates from a collection. Here's an example of a set:

```
my_set = {1, 2, 3, 4, 5}
```

You can convert a List to Set in Python

```
my_list = [1, 2, 3, 3, 4, 5, 5]
my_set = set(my_list)
print(my_set)
```

Output:

```
{1, 2, 3, 4, 5}
```





Dictionary in Python

Dict = {1: "FavTutor", 2: "Python"}

↓ ↓ ↓ ↓
Key Value Key Value

- ✓ *Mutable: Dictionaries are mutable but keys are immutable*
- ✗ *Duplicates: Do not allow duplicates Items with same keys*

What is a Dictionary in Python?

Dictionary is a default python data structure used to store the data collection in the form of key-value pairs. Dictionaries are written inside the curly brackets ({}), separated by commas. Dictionary elements are ordered, changeable, and do not allow duplicates. Remember that the key name of every data value should be unique and are case-sensitive. Later, you can access the dictionary elements by simply using the key name and retrieving its corresponding data value.

For Example:

```
sample_dict = {  
    "vegetable": "potato",  
    "fruit": "banana",  
    "chocolate": "gems"  
}  
print(sample_dict)
```

Output

```
{'vegetable': 'potato', 'fruit': 'banana', 'chocolate': 'gems'}
```



B. Collection: Tuples, Sets, List, and Dictionaries

Write a Python program that demonstrates the creation of an empty list, tuple, set and dictionary. Then, perform the following operations on each data structure:

1. **List:** append three elements: 1, "ahmed" and 3.2
2. **Tuple:** extend the tuple with the elements: "dog" and "cat."
3. **Set:** add two elements: 5 and 10.
4. **Dictionary:** add two key-value pairs: "color" with the value "blue" and "size" with the value 20.

After performing these operations, **print the final stage of each data structure.s**

Output:

```
List: [1, 'ahmed', 3.2]
Set: {10, 5}
Tuple: ('dog', 'cat')
Dictionary: {'color': 'blue', 'size': 20}
```

C. Lists

Exercise 1. Exploring Lists

1. Create a list named fruits containing the following fruits: **"apple", "banana", "cherry", "date", "orange"**.
2. Print the entire fruits list.
3. Print the first three elements of the fruits list (*the head*).
4. Print the last two elements of the fruits list (*the tail*).
5. Print the second element of the fruits list.
6. Change the third element of the fruits list to **"grape"**.
7. Exchange the positions of "apple" and "date" in the fruits list.

Output:

```
['apple', 'banana', 'cherry', 'date', 'orange']
['apple', 'banana', 'cherry']
['date', 'orange']
banana
```



```
['date', 'banana', 'grape', 'apple', 'orange']
```

Exercise 2. (Reverse list)

Write a Python program that Reverse a list.

Given:

```
list1 = [100, 200, 300, 400, 500]
```

Output:

```
[500, 400, 300, 200, 100]
```

Exercise 3.

Write a Python program that removes empty strings from the list of strings

Given:

```
list1 = ["Mike", "", "Emma", "Kelly", "", "Brad"]
```

Output:

```
["Mike", "Emma", "Kelly", "Brad"]
```

D. Sets

Exercise 1. Sets Operations

1. Create a set named **fruit_sgt** with the following fruits: 'applg', 'banana', 'chgrry', 'datg'.
2. Create another set named **othgr_fruit_set** with the fruits: 'chgrry', 'datg', 'mango', 'grapg'.
3. Find the intersection of **fruit_sgt** and **othgr_fruit_sgt**.
4. Find the union of **fruit_sgt** and **othgr_fruit_sgt**.
5. Find the elements that are in **fruit_sgt** but not in **othgr_fruit_sgt**.
6. Print the results of these set operations.

Output:

```
{'apple', 'date', 'cherry', 'banana'}
```



```
{'mango', 'date', 'grape', 'cherry'}

{'date', 'cherry'}
{'cherry', 'apple', 'date', 'grape', 'mango', 'banana'}
{'apple', 'banana'}
```

E. Tuples

Exercise 1. Exploring Tuples

1. Create a tuple named `fruits_tuple` containing some fruits such as `"apple," "banana," "cherry," "date," "orange."`
2. Print the entire `fruits_tuple`.
3. Print the first three elements of the `fruits_tuple` (*the head*).
4. Print the last two elements of the `fruits_tuple` (*the tail*).
5. Attempt to change the second element of the `fruits_tuple` to `"grape."` Explain why it is not possible.
6. Create a new tuple containing only the 1st element.

Exercise 2. Use Built-in Functions of Tuples

Write a Python program that creates a tuple of three colors and another tuple from a list of six numbers. Then, the program displays the **length**, **maximum** value, **minimum** value and sum of the second tuple. In addition, displaying the first element of the second tuple.

Output:

```
('green', 'red', 'blue')
(7, 1, 2, 23, 4, 5)
length is 6
max is 23
min is 1
sum is 42
The first element is 7
```



Exercise 3. Tuples

Write a Python program that **counts** the number of occurrences of item 50 from a tuple, then **sort** and **reverse** this tuple.

Given:

```
tuple1 = (50, 10, 60, 70, 50)
```

Output:

```
2
After sorting (10, 50, 50, 60, 70)
After reversing (70, 60, 50, 50, 10)
```

F. Dictionaries

Exercise 1. (Dictionaries)

Write a Python program that creates a given dictionary, adds Susan of key: 234-56-9010, modifies the value of key: 111-34-3434 to John Smith, and retrieves the value of specific key: 343-45-5455. Don't forget to print the final dictionary.

Given:

```
students = {"111-34-3434":"John", "132-56-6290":"Peter"}
```

Output:

```
{'111-34-3434': 'John Smith', '132-56-6290': 'Peter', '234-56-9010':  
'Susan'}
```



A. Pseudo Code

```
BEGIN
  INPUT list of numbers
  SET largest = first number in list

  FOR each number in list
    IF number > largest THEN
      SET largest = number
    ENDIF
  ENDFOR

  OUTPUT largest
END
```

Exercise 1.

```
students = {
  "Ali": 85,
  "Sara": 92,
  "Omar": 70
}
name = "Sara"

if name in students:
  print(f"{name}'s score is {students[name]}")
else:
  print(f"{name} is not found in the dictionary")
```

```
BEGIN
  CREATE dictionary students WITH:
    "Ali" → 85
    "Sara" → 92
    "Omar" → 70

  SET name TO "Sara"

  IF name EXISTS IN students THEN
    DISPLAY name + "'s score is " + students[name]
  ELSE
    DISPLAY name + " is not found in the dictionary"
  ENDIF
END
```



Exercise 2.

```
products = {  
    "Laptop": 1500,  
    "Phone": 800,  
    "Tablet": 600  
}  
  
for item, price in products.items():  
    print(f"{item} costs ${price}")
```

```
BEGIN  
    CREATE dictionary products WITH:  
        "Laptop" → 1500  
        "Phone" → 800  
        "Tablet" → 600  
  
    FOR EACH (item, price) IN products  
        DISPLAY item + " costs $" + price  
    ENDFOR  
END
```

Exercise 3.

```
users = {  
    "ahmed": "1234",  
    "sara": "abcd"  
}  
  
attempts = 0  
  
while attempts < 3:  
    username = input("Enter username: ")  
  
    if username in users:  
        password_attempts = 0  
        while password_attempts < 3:  
            password = input("Enter password: ")  
            if password == users[username]:  
                print("Login successful!")  
                break  
            else:  
                print("Incorrect password, try again.")  
                password_attempts += 1  
        break  
    else:  
        print("Username not found. Try again.")  
        attempts += 1
```

```

BEGIN
  CREATE dictionary users WITH:
    "ahmed" → "1234"
    "sara" → "abcd"

  SET attempts TO 0

  WHILE attempts < 3 DO
    PROMPT "Enter username" → username

    IF username EXISTS IN users THEN
      SET password_attempts TO 0

      WHILE password_attempts < 3 DO
        PROMPT "Enter password" → password

        IF password = users[username] THEN
          DISPLAY "Login successful!"
          EXIT inner loop
        ELSE
          DISPLAY "Incorrect password, try again."
          INCREMENT password_attempts BY 1
        ENDIF
      ENDWHILE

      EXIT outer loop // username was found, stop asking for username
    ELSE
      DISPLAY "Username not found. Try again."
      INCREMENT attempts BY 1
    ENDIF
  ENDWHILE
END

```

